Probabilistic Analysis, Randomized Algorithms, and Quicksort

Dr. Bugra Caskurlu[1]

[1] School of Computing
New Uzbekistan University

September 22, 2025

Hiring Problem

Description

- You need to hire a new office assistant, and the employment agency will send you one candidate each day.

Hiring Problem

Description

- You need to hire a new office assistant, and the employment agency will send you one candidate each day.
- You will interview that person and then decide to either hire this person or not. You must pay the employment agency a small fee to interview an applicant.

Hiring Problem

Description

- You need to hire a new office assistant, and the employment agency will send you one candidate each day.
- You will interview that person and then decide to either hire this person or not. You must pay the employment agency a small fee to interview an applicant.
- To actually hire an applicant is more costly since you must fire your current office assistant and pay a large hiring fee to the employment agency.

Hiring Problem

Description

- You need to hire a new office assistant, and the employment agency will send you one candidate each day.
- You will interview that person and then decide to either hire this person or not. You must pay the employment agency a small fee to interview an applicant.
- To actually hire an applicant is more costly since you must fire your current office assistant and pay a large hiring fee to the employment agency.
- You are committed to having at all times, the best possible person for the job. Therefore, you decide that, after interviewing each applicant, if that applicant is better qualified than the current office assistant, you will fire the current office assistant and hire the new applicant.

Hiring Problem

Description

- You need to hire a new office assistant, and the employment agency will send you one candidate each day.
- You will interview that person and then decide to either hire this person or not. You must pay the employment agency a small fee to interview an applicant.
- To actually hire an applicant is more costly since you must fire your current office assistant and pay a large hiring fee to the employment agency.
- You are committed to having at all times, the best possible person for the job. Therefore, you decide that, after interviewing each applicant, if that applicant is better qualified than the current office assistant, you will fire the current office assistant and hire the new applicant.
- You are willing to pay the resulting price of this strategy, but you want to estimate what the price will be.

Hiring Problem

Description

- You need to hire a new office assistant, and the employment agency will send you one candidate each day.
- You will interview that person and then decide to either hire this person or not. You must pay the employment agency a small fee to interview an applicant.
- To actually hire an applicant is more costly since you must fire your current office assistant and pay a large hiring fee to the employment agency.
- You are committed to having at all times, the best possible person for the job. Therefore, you decide that, after interviewing each applicant, if that applicant is better qualified than the current office assistant, you will fire the current office assistant and hire the new applicant.
- You are willing to pay the resulting price of this strategy, but you want to estimate what the price will be.
- The total cost: $O(n \cdot c_i + m \cdot c_h)$, where m is the number of people hired.

Hiring Problem

Description

- You need to hire a new office assistant, and the employment agency will send you one candidate each day.
- You will interview that person and then decide to either hire this person or not. You must pay the employment agency a small fee to interview an applicant.
- To actually hire an applicant is more costly since you must fire your current office assistant and pay a large hiring fee to the employment agency.
- You are committed to having at all times, the best possible person for the job. Therefore, you decide that, after interviewing each applicant, if that applicant is better qualified than the current office assistant, you will fire the current office assistant and hire the new applicant.
- You are willing to pay the resulting price of this strategy, but you want to estimate what the price will be.
- The total cost: $O(n \cdot c_i + m \cdot c_h)$, where m is the number of people hired.
- **Worst-Case Analysis:** $O(n \cdot c_h)$.

The Hiring Problem

Pseudocode

```
HIRE-ASSISTANT(C[1 .. n])
    best = 0;
    for i = 1 to n
        interview candidate i;
        if candidate i is better than candidate best
            best = i;
            hire candidate i;
```

Probabilistic Analysis

Motivation

- **Probabilistic analysis** is the use of probability in the analysis of problems (commonly, analysis of running-time).

Probabilistic Analysis

Motivation

- **Probabilistic analysis** is the use of probability in the analysis of problems (commonly, analysis of running-time).
- In order to perform it, we must use knowledge of, or make assumptions about, the **distribution of the inputs**.

Probabilistic Analysis

Motivation

- **Probabilistic analysis** is the use of probability in the analysis of problems (commonly, analysis of running-time).
- In order to perform it, we must use knowledge of, or make assumptions about, the **distribution of the inputs**.
- We assume that we can compare any two candidates and decide which one is better qualified, thus, *there is a total order on the candidates*.

Probabilistic Analysis

Motivation

- **Probabilistic analysis** is the use of probability in the analysis of problems (commonly, analysis of running-time).
- In order to perform it, we must use knowledge of, or make assumptions about, the **distribution of the inputs**.
- We assume that we can compare any two candidates and decide which one is better qualified, thus, *there is a total order on the candidates*.
- We can therefore rank each candidate with a unique number between 1 and n, and adopt the convention that a higher rank corresponds to a better qualified candidate.

Probabilistic Analysis

Motivation

- **Probabilistic analysis** is the use of probability in the analysis of problems (commonly, analysis of running-time).
- In order to perform it, we must use knowledge of, or make assumptions about, the **distribution of the inputs**.
- We assume that we can compare any two candidates and decide which one is better qualified, thus, *there is a total order on the candidates*.
- We can therefore rank each candidate with a unique number between 1 and n, and adopt the convention that a higher rank corresponds to a better qualified candidate.
- Our input is then a permutation of the list $< 1, 2, \ldots, n >$.

Probabilistic Analysis

Motivation

- **Probabilistic analysis** is the use of probability in the analysis of problems (commonly, analysis of running-time).
- In order to perform it, we must use knowledge of, or make assumptions about, the **distribution of the inputs**.
- We assume that we can compare any two candidates and decide which one is better qualified, thus, *there is a total order on the candidates*.
- We can therefore rank each candidate with a unique number between 1 and n, and adopt the convention that a higher rank corresponds to a better qualified candidate.
- Our input is then a permutation of the list $< 1, 2, \ldots, n >$. We say that ranks form a **uniform random permutation** if the list of ranks is **equally likely** to be any of the $n!$ permutations of the numbers 1 through n.

Probabilistic Analysis

Motivation

- **Probabilistic analysis** is the use of probability in the analysis of problems (commonly, analysis of running-time).
- In order to perform it, we must use knowledge of, or make assumptions about, the **distribution of the inputs**.
- We assume that we can compare any two candidates and decide which one is better qualified, thus, *there is a total order on the candidates*.
- We can therefore rank each candidate with a unique number between 1 and n, and adopt the convention that a higher rank corresponds to a better qualified candidate.
- Our input is then a permutation of the list $< 1, 2, \ldots, n >$. We say that ranks form a **uniform random permutation** if the list of ranks is **equally likely** to be any of the $n!$ permutations of the numbers 1 through n.
- Let's **assume** that the ranks form a uniform random permutation and make a probabilistic analysis of the Hire-Assistant algorithm.

Expected Running Time

An Attempt

Let X be the random variable whose value equals the number of times we hire a new office assistant. Then our expected hiring cost is $O(\mathbb{E}[X] \cdot c_h)$, where $\mathbb{E}[X] = \sum_{x=1}^{n} x \cdot Pr\{X = x\}$.

Expected Running Time

An Attempt

Let X be the random variable whose value equals the number of times we hire a new office assistant. Then our expected hiring cost is $O(\mathbb{E}[X] \cdot c_h)$, where $\mathbb{E}[X] = \sum_{x=1}^{n} x \cdot Pr\{X = x\}$. We are done, if we can compute $\mathbb{E}[X]$! But this calculation is cumbersome, we instead will make use of indicator random variables.

Expected Running Time

An Attempt

Let X be the random variable whose value equals the number of times we hire a new office assistant. Then our expected hiring cost is $O(\mathbb{E}[X] \cdot c_h)$, where $\mathbb{E}[X] = \sum_{x=1}^{n} x \cdot Pr\{X = x\}$. We are done, if we can compute $\mathbb{E}[X]$! But this calculation is cumbersome, we instead will make use of indicator random variables.

Indicator Random Variable Definition

Suppose we are given a sample space S and an event A. Then the indicator random variable $I\{A\}$ associated with event A is defined as

$$I\{A\} = \begin{cases} 1 & \text{if } A \text{ occurs,} \\ 0 & \text{if } A \text{ does not occur.} \end{cases}$$

Expected Running Time

An Attempt

Let X be the random variable whose value equals the number of times we hire a new office assistant. Then our expected hiring cost is $O(\mathbb{E}[X] \cdot c_h)$, where $\mathbb{E}[X] = \sum_{x=1}^{n} x \cdot Pr\{X = x\}$. We are done, if we can compute $\mathbb{E}[X]$! But this calculation is cumbersome, we instead will make use of indicator random variables.

Indicator Random Variable Definition

Suppose we are given a sample space S and an event A. Then the indicator random variable $I\{A\}$ associated with event A is defined as

$$I\{A\} = \begin{cases} 1 & \text{if } A \text{ occurs,} \\ 0 & \text{if } A \text{ does not occur.} \end{cases}$$

Lemma

Given a sample space S and an event A in the sample space S, let $X_A = I\{A\}$. Then $\mathbb{E}[X_A] = Pr\{A\}$.

How to Use Indicator Random Variables

Strategy

- **Goal:** Computing the expectation of a random variable. ($\mathbb{E}[X] = \sum_{x=1}^{n} x \cdot Pr\{X = x\}$, where X is a random variable whose value equals the number of times a new assistant is hired.)

How to Use Indicator Random Variables

Strategy

- **Goal:** Computing the expectation of a random variable. ($\mathbb{E}[X] = \sum_{x=1}^{n} x \cdot Pr\{X = x\}$, where X is a random variable whose value equals the number of times a new assistant is hired.)
- **Strategy:** Express the random variable X as the **sum** of a bunch of indicator random variables. Then use **linearity of expectations** to obtain the expected value of X.

How to Use Indicator Random Variables

Strategy

- **Goal:** Computing the expectation of a random variable. ($\mathbb{E}[X] = \sum_{x=1}^{n} x \cdot Pr\{X = x\}$, where X is a random variable whose value equals the number of times a new assistant is hired.)
- **Strategy:** Express the random variable X as the **sum** of a bunch of indicator random variables. Then use **linearity of expectations** to obtain the expected value of X.

Analysis of Hire-Assistant

- Let X_i be the indicator random variable associated with the event in which the ith candidate is hired.

How to Use Indicator Random Variables

Strategy

- **Goal:** Computing the expectation of a random variable. ($\mathbb{E}[X] = \sum_{x=1}^{n} x \cdot Pr\{X = x\}$, where X is a random variable whose value equals the number of times a new assistant is hired.)
- **Strategy:** Express the random variable X as the **sum** of a bunch of indicator random variables. Then use **linearity of expectations** to obtain the expected value of X.

Analysis of Hire-Assistant

- Let X_i be the indicator random variable associated with the event in which the ith candidate is hired.
- Then $X = X_1 + X_2 + \ldots + X_n$.

How to Use Indicator Random Variables

Strategy

- **Goal:** Computing the expectation of a random variable. ($\mathbb{E}[X] = \sum_{x=1}^{n} x \cdot Pr\{X = x\}$, where X is a random variable whose value equals the number of times a new assistant is hired.)
- **Strategy:** Express the random variable X as the **sum** of a bunch of indicator random variables. Then use **linearity of expectations** to obtain the expected value of X.

Analysis of Hire-Assistant

- Let X_i be the indicator random variable associated with the event in which the ith candidate is hired.
- Then $X = X_1 + X_2 + \ldots + X_n$. By linearity of expectations, we have $\mathbb{E}[X] = \sum_{i=1}^{n} \mathbb{E}[X_i]$.

How to Use Indicator Random Variables

Strategy

- **Goal:** Computing the expectation of a random variable. ($\mathbb{E}[X] = \sum_{x=1}^{n} x \cdot Pr\{X = x\}$, where X is a random variable whose value equals the number of times a new assistant is hired.)
- **Strategy:** Express the random variable X as the **sum** of a bunch of indicator random variables. Then use **linearity of expectations** to obtain the expected value of X.

Analysis of Hire-Assistant

- Let X_i be the indicator random variable associated with the event in which the ith candidate is hired.
- Then $X = X_1 + X_2 + \ldots + X_n$. By linearity of expectations, we have $\mathbb{E}[X] = \sum_{i=1}^{n} \mathbb{E}[X_i]$.
- What is $\mathbb{E}[X_i]$?

How to Use Indicator Random Variables

Strategy

- **Goal:** Computing the expectation of a random variable. ($\mathbb{E}[X] = \sum_{x=1}^{n} x \cdot Pr\{X = x\}$, where X is a random variable whose value equals the number of times a new assistant is hired.)
- **Strategy:** Express the random variable X as the **sum** of a bunch of indicator random variables. Then use **linearity of expectations** to obtain the expected value of X.

Analysis of Hire-Assistant

- Let X_i be the indicator random variable associated with the event in which the ith candidate is hired.
- Then $X = X_1 + X_2 + \ldots + X_n$. By linearity of expectations, we have $\mathbb{E}[X] = \sum_{i=1}^{n} \mathbb{E}[X_i]$.
- What is $\mathbb{E}[X_i]$? $\mathbb{E}[X_i] = \frac{1}{i}$.

How to Use Indicator Random Variables

Strategy

- **Goal:** Computing the expectation of a random variable. ($\mathbb{E}[X] = \sum_{x=1}^{n} x \cdot Pr\{X = x\}$, where X is a random variable whose value equals the number of times a new assistant is hired.)
- **Strategy:** Express the random variable X as the **sum** of a bunch of indicator random variables. Then use **linearity of expectations** to obtain the expected value of X.

Analysis of Hire-Assistant

- Let X_i be the indicator random variable associated with the event in which the ith candidate is hired.
- Then $X = X_1 + X_2 + \ldots + X_n$. By linearity of expectations, we have $\mathbb{E}[X] = \sum_{i=1}^{n} \mathbb{E}[X_i]$.
- What is $\mathbb{E}[X_i]$? $\mathbb{E}[X_i] = \frac{1}{i}$. Thus, $\mathbb{E}[X] = \sum_{i=1}^{n} \frac{1}{i} = \ln n + O(1)$.

How to Use Indicator Random Variables

Strategy

- **Goal:** Computing the expectation of a random variable. ($\mathbb{E}[X] = \sum_{x=1}^{n} x \cdot Pr\{X = x\}$, where X is a random variable whose value equals the number of times a new assistant is hired.)
- **Strategy:** Express the random variable X as the **sum** of a bunch of indicator random variables. Then use **linearity of expectations** to obtain the expected value of X.

Analysis of Hire-Assistant

- Let X_i be the indicator random variable associated with the event in which the ith candidate is hired.
- Then $X = X_1 + X_2 + \ldots + X_n$. By linearity of expectations, we have $\mathbb{E}[X] = \sum_{i=1}^{n} \mathbb{E}[X_i]$.
- What is $\mathbb{E}[X_i]$? $\mathbb{E}[X_i] = \frac{1}{i}$. Thus, $\mathbb{E}[X] = \sum_{i=1}^{n} \frac{1}{i} = \ln n + O(1)$.
- Thus, **average** hiring cost of the Hire-Assistant algorithm, **assuming** ranks form a uniform random permutation is $O(n \cdot c_i + \ln n \cdot c_h)$.

Randomized Algorithms

Motivation

- Knowing the distribution of the inputs can help analyzing the average-case behaviour of an algorithm.

Randomized Algorithms

Motivation

- Knowing the distribution of the inputs can help analyzing the average-case behaviour of an algorithm.
- Often the time, we do not have such knowledge and no average-case is possible.

Randomized Algorithms

Motivation

- Knowing the distribution of the inputs can help analyzing the average-case behaviour of an algorithm.
- Often the time, we do not have such knowledge and no average-case is possible. Sometimes we may have such a knowledge but the distribution may make it hard to analyze.

Randomized Algorithms

Motivation

- Knowing the distribution of the inputs can help analyzing the average-case behaviour of an algorithm.
- Often the time, we do not have such knowledge and no average-case is possible. Sometimes we may have such a knowledge but the distribution may make it hard to analyze.
- What if we can impose a distribution on the input?

Randomized Algorithms

Motivation

- Knowing the distribution of the inputs can help analyzing the average-case behaviour of an algorithm.
- Often the time, we do not have such knowledge and no average-case is possible. Sometimes we may have such a knowledge but the distribution may make it hard to analyze.
- What if we can impose a distribution on the input?
- Let's change the model slightly. We will say that the employment agency has n candidates, and they send us a list of the candidates in advance. On each day, we choose, randomly, which candidate to interview.

Randomized Algorithms

Motivation

- Knowing the distribution of the inputs can help analyzing the average-case behaviour of an algorithm.
- Often the time, we do not have such knowledge and no average-case is possible. Sometimes we may have such a knowledge but the distribution may make it hard to analyze.
- What if we can impose a distribution on the input?
- Let's change the model slightly. We will say that the employment agency has n candidates, and they send us a list of the candidates in advance. On each day, we choose, randomly, which candidate to interview.
- Although we know nothing about the candidates (besides their names), we have made a significant change. Instead of relying on a guess that the candidates will come to us in random order, we have instead gained control of the process and enforced a random order.

Randomized Algorithms

Motivation

- Knowing the distribution of the inputs can help analyzing the average-case behaviour of an algorithm.
- Often the time, we do not have such knowledge and no average-case is possible. Sometimes we may have such a knowledge but the distribution may make it hard to analyze.
- What if we can impose a distribution on the input?
- Let's change the model slightly. We will say that the employment agency has n candidates, and they send us a list of the candidates in advance. On each day, we choose, randomly, which candidate to interview.
- Although we know nothing about the candidates (besides their names), we have made a significant change. Instead of relying on a guess that the candidates will come to us in random order, we have instead gained control of the process and enforced a random order.
- We call an algorithm **randomized** if its behavior is determined not only by its input but also by values produced by a random number generator.

Randomized Hiring Problem

Pseudocode

```
RANDOMIZED-HIRE-ASSISTANT(C[1 .. n])
    randomly permute the list of candidates
    best = 0;
    for i = 1 to n
        interview candidate i;
        if candidate i is better than candidate best
            best = i;
            hire candidate i;
```

Randomized Hiring Problem

Pseudocode

```
RANDOMIZED-HIRE-ASSISTANT(C[1 .. n])
    randomly permute the list of candidates
    best = 0;
    for i = 1 to n
        interview candidate i;
        if candidate i is better than candidate best
            best = i;
            hire candidate i;
```

Expected Cost

Thus, **expected** hiring cost of the Randomized-Hire-Assistant algorithm is $O(n \cdot c_i + \ln n \cdot c_h)$.

Randomize In-Place

Pseudocode

```
RANDOMIZE-IN-PLACE(A)
    n = length[A];
    for i = 1 to n
        SWAP A[i] with A[RANDOM(i,n)];
```

Randomize In-Place

Pseudocode

```
RANDOMIZE-IN-PLACE(A)
    n = length[A];
    for i = 1 to n
        SWAP A[i] with A[RANDOM(i,n)];
```

Expected Cost

- Procedure **SWAP** swaps its two inputs.
- Procedure **RANDOM** takes two integers a and b as parameters and returns an integer between a and b, where the probability of each number in the range to be returned is $\frac{1}{b-a+1}$.

Randomize In-Place

Pseudocode

```
RANDOMIZE-IN-PLACE(A)
    n = length[A];
    for i = 1 to n
        SWAP A[i] with A[RANDOM(i,n)];
```

Expected Cost

- Procedure **SWAP** swaps its two inputs.
- Procedure **RANDOM** takes two integers a and b as parameters and returns an integer between a and b, where the probability of each number in the range to be returned is $\frac{1}{b-a+1}$.
- **Loop Invariant:** Just prior to the ith iteration of the for loop, for each possible $(i-1)$-permutation, the subarray $A[1 \ldots (i-1)]$ contains this $(i-1)$-permutation with probability $\frac{(n-i+1)!}{n!}$.

Overview

Sorting Algorithms

- **Insertion Sort:** *Incremental algorithm. Runs in $O(n^2)$-time.* Sorts *in place*.

Overview

Sorting Algorithms

- **Insertion Sort:** *Incremental algorithm. Runs in $O(n^2)$-time.* Sorts *in place*.
- **Merge Sort:** *Divide-and-conquer algorithm. Runs in $\Theta(n \cdot \lg n)$-time.* **Not** an *in place* algorithm, requires $\Theta(n)$ additional space.

Overview

Sorting Algorithms

- **Insertion Sort:** *Incremental algorithm. Runs in $O(n^2)$-time.* Sorts *in place*.
- **Merge Sort:** *Divide-and-conquer algorithm. Runs in $\Theta(n \cdot \lg n)$-time.* **Not** an *in place* algorithm, requires $\Theta(n)$ additional space.
- **Heapsort:** *Uses heaps. Runs in $O(n \lg n)$-time*, and an *in place* algorithm.

Overview

Sorting Algorithms

- **Insertion Sort:** *Incremental algorithm. Runs in $O(n^2)$-time.* Sorts *in place*.
- **Merge Sort:** *Divide-and-conquer algorithm. Runs in $\Theta(n \cdot \lg n)$-time.* **Not** an *in place* algorithm, requires $\Theta(n)$ additional space.
- **Heapsort:** *Uses heaps. Runs in $O(n \lg n)$-time*, and an *in place* algorithm.
- Is it possible to improve the running-time further?

Overview

Sorting Algorithms

- **Insertion Sort:** *Incremental algorithm. Runs in $O(n^2)$-time.* Sorts *in place*.
- **Merge Sort:** *Divide-and-conquer algorithm. Runs in $\Theta(n \cdot \lg n)$-time.* **Not** an *in place* algorithm, requires $\Theta(n)$ additional space.
- **Heapsort:** *Uses heaps. Runs in $O(n \lg n)$-time*, and an *in place* algorithm.
- Is it possible to improve the running-time further? Not quite, unless we have some assumptions on the input.

Overview

Sorting Algorithms

- **Insertion Sort:** *Incremental algorithm. Runs in $O(n^2)$-time.* Sorts *in place*.
- **Merge Sort:** *Divide-and-conquer algorithm. Runs in $\Theta(n \cdot \lg n)$-time.* **Not** an *in place* algorithm, requires $\Theta(n)$ additional space.
- **Heapsort:** *Uses heaps. Runs in $O(n \lg n)$-time*, and an *in place* algorithm.
- Is it possible to improve the running-time further? Not quite, unless we have some assumptions on the input. Every **comparison based sorting algorithm will take $\Omega(n \cdot \lg n)$-time in the worst case**.

Overview

Sorting Algorithms

- **Insertion Sort:** *Incremental algorithm. Runs in $O(n^2)$-time.* Sorts *in place*.
- **Merge Sort:** *Divide-and-conquer algorithm. Runs in $\Theta(n \cdot \lg n)$-time.* **Not** an *in place* algorithm, requires $\Theta(n)$ additional space.
- **Heapsort:** *Uses heaps. Runs in $O(n \lg n)$-time*, and an *in place* algorithm.
- Is it possible to improve the running-time further? Not quite, unless we have some assumptions on the input. Every **comparison based sorting algorithm will take $\Omega(n \cdot \lg n)$-time in the worst case**.
- **QuickSort:** An in place algorithm that runs in $\Theta(n^2)$-time in the worst case.

Overview

Sorting Algorithms

- **Insertion Sort:** *Incremental algorithm. Runs in $O(n^2)$-time.* Sorts *in place*.
- **Merge Sort:** *Divide-and-conquer algorithm. Runs in $\Theta(n \cdot \lg n)$-time.* **Not** an *in place* algorithm, requires $\Theta(n)$ additional space.
- **Heapsort:** *Uses heaps. Runs in $O(n \lg n)$-time*, and an *in place* algorithm.
- Is it possible to improve the running-time further? Not quite, unless we have some assumptions on the input. Every **comparison based sorting algorithm will take $\Omega(n \cdot \lg n)$-time in the worst case**.
- **QuickSort:** An in place algorithm that runs in $\Theta(n^2)$-time in the worst case. However, it is remarkable efficient on the ”average”: its ”average-case” running time is $\Theta(n \cdot \lg n)$, and the constants hidden in the $\Theta(n \cdot \lg n)$ notation are quite small. Thus, it is often the best choice for sorting in practice.

Description

Quick Sort

- Quicksort is based on the **divide and conquer paradigm**.

Description

Quick Sort

- Quicksort is based on the **divide and conquer paradigm**.
- **Divide:** Partition (rearrange) the array $A[p \ldots r]$ into two (possibly empty) subarrays $A[p \ldots (q-1)]$ and $A[(q+1) \ldots r]$ such that each element of $A[p \ldots (q-1)]$ is less than or equal to $A[q]$, which is, in turn, less than or equal to each element of $A[(q+1) \ldots r]$. We also compute the index q as part of the partitioning process.

Description

Quick Sort

- Quicksort is based on the **divide and conquer paradigm**.
- **Divide:** Partition (rearrange) the array $A[p \ldots r]$ into two (possibly empty) subarrays $A[p \ldots (q-1)]$ and $A[(q+1) \ldots r]$ such that each element of $A[p \ldots (q-1)]$ is less than or equal to $A[q]$, which is, in turn, less than or equal to each element of $A[(q+1) \ldots r]$. We also compute the index q as part of the partitioning process.
- **Conquer:** Sort the two subarrays $A[p \ldots (q-1)]$ and $A[(q+1) \ldots r]$ by recursive calls to quicksort.

Description

Quick Sort

- Quicksort is based on the **divide and conquer paradigm**.
- **Divide:** Partition (rearrange) the array $A[p \ldots r]$ into two (possibly empty) subarrays $A[p \ldots (q-1)]$ and $A[(q+1) \ldots r]$ such that each element of $A[p \ldots (q-1)]$ is less than or equal to $A[q]$, which is, in turn, less than or equal to each element of $A[(q+1) \ldots r]$. We also compute the index q as part of the partitioning process.
- **Conquer:** Sort the two subarrays $A[p \ldots (q-1)]$ and $A[(q+1) \ldots r]$ by recursive calls to quicksort.
- **Combine:** Since the subarrays are sorted in place, no work is needed to combine them: the entire array $A[p \ldots r]$ is now sorted.

QuickSort Algorithm

Pseudocode

```
QUICKSORT (A, p, r)
    if p < r
        q = PARTITION (A, p, r);
        QUICKSORT (A, p, q - 1);
        QUICKSORT (A, q + 1, r);
```

QuickSort Algorithm

Pseudocode

```
QUICKSORT (A, p, r)
    if p < r
        q = PARTITION (A, p, r);
        QUICKSORT (A, p, q - 1);
        QUICKSORT (A, q + 1, r);
```

Initial Call

To sort an entire array *A*, the initial call is **QUICKSORT** (A, 1, *length*[A]).

Partitioning the Array

Pseudocode

```
PARTITION (A, p, r)
    x = A[r];
    i = p - 1;

    for j = p to r - 1
        if A[j] ≤ x
            i = i + 1;
            exchange A[i] ↔ A[j];

    exchange A[i + 1] ↔ A[r];
    return (i + 1);
```

Partitioning the Array

Pseudocode

```
PARTITION (A, p, r)
    x = A[r];
    i = p - 1;

    for j = p to r - 1
        if A[j] ≤ x
            i = i + 1;
            exchange A[i] ↔ A[j];

    exchange A[i + 1] ↔ A[r];
    return (i + 1);
```

Loop Invariant

At the beginning of each iteration of the loop, for any array index k,

- If $p \leq k \leq i$, then $A[k] \leq x$.
- If $(i+1) \leq k \leq (j-1)$, then $A[k] > x$.
- If $k = r$, then $A[k] = x$.

Proving Invariant

Loop Invariant

At the beginning of each iteration of the loop, for any array index k,

- If $p \leq k \leq i$, then $A[k] \leq x$.
- If $(i+1) \leq k \leq (j-1)$, then $A[k] > x$.
- If $k = r$, then $A[k] = x$.

Proving Invariant

Loop Invariant

At the beginning of each iteration of the loop, for any array index k,

- If $p \leq k \leq i$, then $A[k] \leq x$.
- If $(i+1) \leq k \leq (j-1)$, then $A[k] > x$.
- If $k = r$, then $A[k] = x$.

Initialization

Prior to the first iteration of the loop, $i = p-1$, and $j = p$. There are no values between p and i, and no values between $i+1$ and $j-1$, so the first conditions of the loop invariant are trivially satisfied. The assignment in line 1 of the pseudocode satisfies the third condition.

Proving Invariant Cont.

Loop Invariant

At the beginning of each iteration of the loop, for any array index k,

- If $p \leq k \leq i$, then $A[k] \leq x$.
- If $(i+1) \leq k \leq (j-1)$, then $A[k] > x$.
- If $k = r$, then $A[k] = x$.

Proving Invariant Cont.

Loop Invariant

At the beginning of each iteration of the loop, for any array index k,

- If $p \leq k \leq i$, then $A[k] \leq x$.
- If $(i+1) \leq k \leq (j-1)$, then $A[k] > x$.
- If $k = r$, then $A[k] = x$.

Maintenance

There are two cases to consider. If $A[j] > x$, the only action in the loop is to increment j. After j is incremented, condition 2 holds for $A[j-1]$ and all other entries remain unchanged.

Proving Invariant Cont.

Loop Invariant

At the beginning of each iteration of the loop, for any array index k,

- If $p \leq k \leq i$, then $A[k] \leq x$.
- If $(i+1) \leq k \leq (j-1)$, then $A[k] > x$.
- If $k = r$, then $A[k] = x$.

Maintenance

There are two cases to consider. If $A[j] > x$, the only action in the loop is to increment j. After j is incremented, condition 2 holds for $A[j-1]$ and all other entries remain unchanged.If $A[j] \leq x$; i is incremented, $A[i]$ and $A[j]$ are swapped, and then j is incremented. Because of the swap, we now have $A[i] \leq x$, and condition 1 is satisfied. Similarly we have $A[j-1] > x$, since the item that was swapped into $A[j-1]$ is, by the loop invariant, greater than x.

Termination Condition and Analysis

Termination

At termination, $j = r$. Therefore, every entry in the array is one of the three sets described by the invariant, and we have partitioned the values in the array into three sets: those less than equal to x, those greater than or equal to x, and a singleton containing x.

Termination Condition and Analysis

Termination

At termination, $j = r$. Therefore, every entry in the array is one of the three sets described by the invariant, and we have partitioned the values in the array into three sets: those less than equal to x, those greater than or equal to x, and a singleton containing x.

Analysis

- The final two lines of **PARTITION** move the pivot element into its place in the middle of the array by swapping it with the leftmost element that is greater than x.

Termination Condition and Analysis

Termination

At termination, $j = r$. Therefore, every entry in the array is one of the three sets described by the invariant, and we have partitioned the values in the array into three sets: those less than equal to x, those greater than or equal to x, and a singleton containing x.

Analysis

- The final two lines of **PARTITION** move the pivot element into its place in the middle of the array by swapping it with the leftmost element that is greater than x.The output now satisfies the specifications for the given divide step.

Termination Condition and Analysis

Termination

At termination, $j = r$. Therefore, every entry in the array is one of the three sets described by the invariant, and we have partitioned the values in the array into three sets: those less than equal to x, those greater than or equal to x, and a singleton containing x.

Analysis

- The final two lines of **PARTITION** move the pivot element into its place in the middle of the array by swapping it with the leftmost element that is greater than x.The output now satisfies the specifications for the given divide step.
- The running-time of **PARTITION** is $\Theta(n)$, where $n = r - p + 1$.

Performance

Partitioning Discussions

- The running-time of quicksort depends on whether the partitioning is balanced or unbalanced, and this in turn depends on which elements are used as pivot for partitioning.

Performance

Partitioning Discussions

- The running-time of quicksort depends on whether the partitioning is balanced or unbalanced, and this in turn depends on which elements are used as pivot for partitioning.
- If the partitioning is balanced, the algorithm runs asymptotically as fast as merge sort.

Performance

Partitioning Discussions

- The running-time of quicksort depends on whether the partitioning is balanced or unbalanced, and this in turn depends on which elements are used as pivot for partitioning.
- If the partitioning is balanced, the algorithm runs asymptotically as fast as merge sort.
- If the partitioning is unbalanced, it can run asymptotically as slowly as insertion sort.

Performance

Partitioning Discussions

- The running-time of quicksort depends on whether the partitioning is balanced or unbalanced, and this in turn depends on which elements are used as pivot for partitioning.
- If the partitioning is balanced, the algorithm runs asymptotically as fast as merge sort.
- If the partitioning is unbalanced, it can run asymptotically as slowly as insertion sort.

Worst-Case Partitioning

The worst-case behavior for quicksort occurs when the partitioning routine produces one subproblem with $n-1$ elements and one with 0 elements.

Performance

Partitioning Discussions

- The running-time of quicksort depends on whether the partitioning is balanced or unbalanced, and this in turn depends on which elements are used as pivot for partitioning.
- If the partitioning is balanced, the algorithm runs asymptotically as fast as merge sort.
- If the partitioning is unbalanced, it can run asymptotically as slowly as insertion sort.

Worst-Case Partitioning

The worst-case behavior for quicksort occurs when the partitioning routine produces one subproblem with $n-1$ elements and one with 0 elements. In this case, the recurrence for the running-time is $T(n) = T(n-1) + T(0) + \Theta(n) = T(n-1) + \Theta(n)$. The solution to this recurrence is $\Theta(n^2)$.

Partitioning Discussion

Best-Case Partitioning

- In the most even possible split, **PARTITION** produces two subproblems, each of which size no more than $\frac{n}{2}$.

Partitioning Discussion

Best-Case Partitioning

- In the most even possible split, **PARTITION** produces two subproblems, each of which size no more than $\frac{n}{2}$.
- Then the runtime can be described by the recurrence $T(n) \leq 2 \cdot T(\frac{n}{2}) + \Theta(n)$, the solution of which is $O(n \cdot \lg n)$.

Partitioning Discussion

Best-Case Partitioning

- In the most even possible split, **PARTITION** produces two subproblems, each of which size no more than $\frac{n}{2}$.
- Then the runtime can be described by the recurrence $T(n) \leq 2 \cdot T(\frac{n}{2}) + \Theta(n)$, the solution of which is $O(n \cdot \lg n)$.

Partitioning Discussion

Best-Case Partitioning

- In the most even possible split, **PARTITION** produces two subproblems, each of which size no more than $\frac{n}{2}$.
- Then the runtime can be described by the recurrence $T(n) \leq 2 \cdot T(\frac{n}{2}) + \Theta(n)$, the solution of which is $O(n \cdot \lg n)$.

Balanced Partitioning

- Assume the partitioning algorithm produces a 9-to-1 proportional split.

Partitioning Discussion

Best-Case Partitioning

- In the most even possible split, **PARTITION** produces two subproblems, each of which size no more than $\frac{n}{2}$.
- Then the runtime can be described by the recurrence $T(n) \leq 2 \cdot T(\frac{n}{2}) + \Theta(n)$, the solution of which is $O(n \cdot \lg n)$.

Balanced Partitioning

- Assume the partitioning algorithm produces a 9-to-1 proportional split.
- We then have the recurrence $T(n) \leq T(\frac{9n}{10}) + T(\frac{n}{10}) + c \cdot n$.

Partitioning Discussion

Best-Case Partitioning

- In the most even possible split, **PARTITION** produces two subproblems, each of which size no more than $\frac{n}{2}$.
- Then the runtime can be described by the recurrence $T(n) \leq 2 \cdot T(\frac{n}{2}) + \Theta(n)$, the solution of which is $O(n \cdot \lg n)$.

Balanced Partitioning

- Assume the partitioning algorithm produces a 9-to-1 proportional split.
- We then have the recurrence $T(n) \leq T(\frac{9n}{10}) + T(\frac{n}{10}) + c \cdot n$.
- By the recursion tree method, it can be shown that the solution to this recurrence is at most $c \cdot n \cdot \lg_{\frac{10}{9}} n = \Theta(n \cdot \lg n)$.

Pseudocode for Randomized Quicksort

Randomized Partition

```
RANDOMIZED-PARTITION (A, p, r)
    i = RANDOM(p,r);
    exchange A[r] ↔ A[i];
    return PARTITION(A, p, r);
```

Pseudocode for Randomized Quicksort

Randomized Partition

```
RANDOMIZED-PARTITION (A, p, r)
    i = RANDOM(p,r);
    exchange A[r] ↔ A[i];
    return PARTITION(A, p, r);
```

Randomized Quicksort

```
RANDOMIZED-QUICKSORT (A, p, r)
    if p < r
        q = RANDOMIZED-PARTITION (A, p, r);
        RANDOMIZED-QUICKSORT (A, p, q - 1);
        RANDOMIZED-QUICKSORT (A, q + 1, r);
```

Expected Running Time of Quicksort

The Big Picture

- The running time of Quicksort is dominated by the time spent in the **PARTITION** procedure.

Expected Running Time of Quicksort

The Big Picture

- The running time of Quicksort is dominated by the time spent in the **PARTITION** procedure.
- Each time the **PARTITION** procedure is called, a pivot element is selected, and this element is never included in any future recursive calls to **PARTITION** or **QUICKSORT**.

Expected Running Time of Quicksort

The Big Picture

- The running time of Quicksort is dominated by the time spent in the **PARTITION** procedure.
- Each time the **PARTITION** procedure is called, a pivot element is selected, and this element is never included in any future recursive calls to **PARTITION** or **QUICKSORT**.
- Thus, there can be at most n calls to **PARTITION** over the entire execution of the algorithm.

Expected Running Time of Quicksort

The Big Picture

- The running time of Quicksort is dominated by the time spent in the **PARTITION** procedure.
- Each time the **PARTITION** procedure is called, a pivot element is selected, and this element is never included in any future recursive calls to **PARTITION** or **QUICKSORT**.
- Thus, there can be at most n calls to **PARTITION** over the entire execution of the algorithm.
- One call to **PARTITION** takes $O(1)$ time plus an amount of time proportional to the **number of iterations of the for loop** in the body of the **PARTITION**.

Expected Running Time of Quicksort

The Big Picture

- The running time of Quicksort is dominated by the time spent in the **PARTITION** procedure.
- Each time the **PARTITION** procedure is called, a pivot element is selected, and this element is never included in any future recursive calls to **PARTITION** or **QUICKSORT**.
- Thus, there can be at most n calls to **PARTITION** over the entire execution of the algorithm.
- One call to **PARTITION** takes $O(1)$ time plus an amount of time proportional to the **number of iterations of the for loop** in the body of the **PARTITION**.
- *Each iteration of this for loop performs **one** comparison, i.e., comparing the pivot element to another element of the array A.*

Expected Running Time of Randomized Quicksort Cont.

Lemma

Let X be the number of comparisons performed by **PARTITION** over the entire execution of the algorithm of **QUICKSORT** on an array of size n. Then the running time of **QUICKSORT** is $O(X + n)$.

Expected Running Time of Randomized Quicksort Cont.

Lemma

Let X be the number of comparisons performed by **PARTITION** over the entire execution of the algorithm of **QUICKSORT** on an array of size n. Then the running time of **QUICKSORT** is $O(X + n)$.

Analysis Strategy

- Our goal is to compute $\mathbb{E}[X]$, i.e., the expected number of comparisons made performed in all calls to **PARTITION**.

Expected Running Time of Randomized Quicksort Cont.

Lemma

Let X be the number of comparisons performed by **PARTITION** over the entire execution of the algorithm of **QUICKSORT** on an array of size n. Then the running time of **QUICKSORT** is $O(X + n)$.

Analysis Strategy

- Our goal is to compute $\mathbb{E}[X]$, i.e., the expected number of comparisons made performed in all calls to **PARTITION**.
- We will not attempt to analyze how many comparisons are made in each call to **PARTITION**; rather, we will derive an overall bound on the total number of comparisons.

Expected Running Time of Randomized Quicksort Cont.

Lemma

Let X be the number of comparisons performed by **PARTITION** over the entire execution of the algorithm of **QUICKSORT** on an array of size n. Then the running time of **QUICKSORT** is $O(X + n)$.

Analysis Strategy

- Our goal is to compute $\mathbb{E}[X]$, i.e., the expected number of comparisons made performed in all calls to **PARTITION**.
- We will not attempt to analyze how many comparisons are made in each call to **PARTITION**; rather, we will derive an overall bound on the total number of comparisons.
- To do that, we shall understand when the algorithm compares two elements of the array and when it does not!

Expected Running Time of Randomized Quicksort Cont.

Definitions

- For ease of analysis, we rename the elements of the array A as $z_1, z_2, \ldots, z_n$ with z_i being the ith smallest element.

Expected Running Time of Randomized Quicksort Cont.

Definitions

- For ease of analysis, we rename the elements of the array A as $z_1, z_2, \ldots, z_n$ with z_i being the ith smallest element.
- We define the set $Z_{ij} = \{z_i, z_{i+1}, \ldots, z_j\}$ to be the set of elements between z_i and z_j inclusive.

Expected Running Time of Randomized Quicksort Cont.

Definitions

- For ease of analysis, we rename the elements of the array A as $z_1, z_2, \ldots, z_n$ with z_i being the ith smallest element.
- We define the set $Z_{ij} = \{z_i, z_{i+1}, \ldots, z_j\}$ to be the set of elements between z_i and z_j inclusive.

Observation

- When does the algorithm compare z_i and z_j?

Expected Running Time of Randomized Quicksort Cont.

Definitions

- For ease of analysis, we rename the elements of the array A as $z_1, z_2, \ldots, z_n$ with z_i being the ith smallest element.
- We define the set $Z_{ij} = \{z_i, z_{i+1}, \ldots, z_j\}$ to be the set of elements between z_i and z_j inclusive.

Observation

- When does the algorithm compare z_i and z_j?
- Observe that *each pair of elements is compared at most once*. Why?

Expected Running Time of Randomized Quicksort Cont.

Definitions

- For ease of analysis, we rename the elements of the array A as $z_1, z_2, \ldots, z_n$ with z_i being the ith smallest element.
- We define the set $Z_{ij} = \{z_i, z_{i+1}, \ldots, z_j\}$ to be the set of elements between z_i and z_j inclusive.

Observation

- When does the algorithm compare z_i and z_j?
- Observe that *each pair of elements is compared at most once*. Why? Elements are compared only to the pivot element and, after a particular call of **PARTITION** finishes, the pivot element used in that call is never again compared to any other elements.

Analysis

Analysis Using Indicator Random Variables

- We define $X_{ij} = I\{z_i \text{ is compared to } z_j\}$, where we are considering whether the comparison takes place at any time during the execution of the algorithm.

Analysis

Analysis Using Indicator Random Variables

- We define $X_{ij} = I\{z_i \text{ is compared to } z_j\}$, where we are considering whether the comparison takes place at any time during the execution of the algorithm.
- But then $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$!

Analysis

Analysis Using Indicator Random Variables

- We define $X_{ij} = I\{z_i \text{ is compared to } z_j\}$, where we are considering whether the comparison takes place at any time during the execution of the algorithm.
- But then $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$!
- So,
  $\mathbb{E}[X] = \mathbb{E}[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}]$

Analysis

Analysis Using Indicator Random Variables

- We define $X_{ij} = I\{z_i \text{ is compared to } z_j\}$, where we are considering whether the comparison takes place at any time during the execution of the algorithm.
- But then $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$!
- So,
  $\mathbb{E}[X] = \mathbb{E}[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{E}[X_{ij}]$

Analysis

Analysis Using Indicator Random Variables

- We define $X_{ij} = I\{z_i \text{ is compared to } z_j\}$, where we are considering whether the comparison takes place at any time during the execution of the algorithm.
- But then $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$!
- So,
  $\mathbb{E}[X] = \mathbb{E}[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{E}[X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} Pr\{z_i \text{ is compared to } z_j\}$.

Analysis

Analysis Using Indicator Random Variables

- We define $X_{ij} = I\{z_i \text{ is compared to } z_j\}$, where we are considering whether the comparison takes place at any time during the execution of the algorithm.
- But then $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$!
- So,
  $\mathbb{E}[X] = \mathbb{E}[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{E}[X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} Pr\{z_i \text{ is compared to } z_j\}$.
- What is $Pr\{z_i \text{ is compared to } z_j\}$?

Analysis

Analysis Using Indicator Random Variables

- We define $X_{ij} = I\{z_i \text{ is compared to } z_j\}$, where we are considering whether the comparison takes place at any time during the execution of the algorithm.
- But then $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$!
- So,
  $\mathbb{E}[X] = \mathbb{E}[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{E}[X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} Pr\{z_i \text{ is compared to } z_j\}$.
- What is $Pr\{z_i \text{ is compared to } z_j\}$?
- z_i and z_j are compared if and only if z_i or z_j is the first pivot of the set Z_{ij}, and thus $Pr\{z_i \text{ is compared to } z_j\} = \frac{2}{j-i+1}$.

Analysis

Analysis Using Indicator Random Variables

- We define $X_{ij} = I\{z_i \text{ is compared to } z_j\}$, where we are considering whether the comparison takes place at any time during the execution of the algorithm.
- But then $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$!
- So,
  $\mathbb{E}[X] = \mathbb{E}[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{E}[X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} Pr\{z_i \text{ is compared to } z_j\}$.
- What is $Pr\{z_i \text{ is compared to } z_j\}$?
- z_i and z_j are compared if and only if z_i or z_j is the first pivot of the set Z_{ij}, and thus $Pr\{z_i \text{ is compared to } z_j\} = \frac{2}{j-i+1}$.
- Thus,
  $\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1}$

Analysis

Analysis Using Indicator Random Variables

- We define $X_{ij} = I\{z_i \text{ is compared to } z_j\}$, where we are considering whether the comparison takes place at any time during the execution of the algorithm.
- But then $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$!
- So,
  $\mathbb{E}[X] = \mathbb{E}[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{E}[X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} Pr\{z_i \text{ is compared to } z_j\}$.
- What is $Pr\{z_i \text{ is compared to } z_j\}$?
- z_i and z_j are compared if and only if z_i or z_j is the first pivot of the set Z_{ij}, and thus $Pr\{z_i \text{ is compared to } z_j\} = \frac{2}{j-i+1}$.
- Thus,
  $\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1}$

Analysis

Analysis Using Indicator Random Variables

- We define $X_{ij} = I\{z_i \text{ is compared to } z_j\}$, where we are considering whether the comparison takes place at any time during the execution of the algorithm.
- But then $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$!
- So,
  $\mathbb{E}[X] = \mathbb{E}[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{E}[X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} Pr\{z_i \text{ is compared to } z_j\}$.
- What is $Pr\{z_i \text{ is compared to } z_j\}$?
- z_i and z_j are compared if and only if z_i or z_j is the first pivot of the set Z_{ij}, and thus $Pr\{z_i \text{ is compared to } z_j\} = \frac{2}{j-i+1}$.
- Thus,
  $\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} \sum_{k=1}^{n} \frac{2}{k}$

Analysis

Analysis Using Indicator Random Variables

- We define $X_{ij} = I\{z_i \text{ is compared to } z_j\}$, where we are considering whether the comparison takes place at any time during the execution of the algorithm.
- But then $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$!
- So,
  $\mathbb{E}[X] = \mathbb{E}[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{E}[X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} Pr\{z_i \text{ is compared to } z_j\}$.
- What is $Pr\{z_i \text{ is compared to } z_j\}$?
- z_i and z_j are compared if and only if z_i or z_j is the first pivot of the set Z_{ij}, and thus $Pr\{z_i \text{ is compared to } z_j\} = \frac{2}{j-i+1}$.
- Thus,
  $\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} \sum_{k=1}^{n} \frac{2}{k} = \sum_{i=1}^{n-1} O(\lg n)$

Analysis

Analysis Using Indicator Random Variables

- We define $X_{ij} = I\{z_i \text{ is compared to } z_j\}$, where we are considering whether the comparison takes place at any time during the execution of the algorithm.
- But then $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$!
- So,
  $\mathbb{E}[X] = \mathbb{E}[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{E}[X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} Pr\{z_i \text{ is compared to } z_j\}$.
- What is $Pr\{z_i \text{ is compared to } z_j\}$?
- z_i and z_j are compared if and only if z_i or z_j is the first pivot of the set Z_{ij}, and thus $Pr\{z_i \text{ is compared to } z_j\} = \frac{2}{j-i+1}$.
- Thus,
  $\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} \sum_{k=1}^{n} \frac{2}{k} = \sum_{i=1}^{n-1} O(\lg n) = O(n \cdot \lg n)$.

Analysis

Analysis Using Indicator Random Variables

- We define $X_{ij} = I\{z_i \text{ is compared to } z_j\}$, where we are considering whether the comparison takes place at any time during the execution of the algorithm.
- But then $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$!
- So,
  $\mathbb{E}[X] = \mathbb{E}[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{E}[X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} Pr\{z_i \text{ is compared to } z_j\}$.
- What is $Pr\{z_i \text{ is compared to } z_j\}$?
- z_i and z_j are compared if and only if z_i or z_j is the first pivot of the set Z_{ij}, and thus $Pr\{z_i \text{ is compared to } z_j\} = \frac{2}{j-i+1}$.
- Thus,
  $\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} \sum_{k=1}^{n} \frac{2}{k} = \sum_{i=1}^{n-1} O(\lg n) = O(n \cdot \lg n)$.
- Thus, the expected running time of randomized quicksort (or average-case running time of quicksort assuming that the input array is a uniform random permutation) is $O(n \cdot \lg n)$.